04

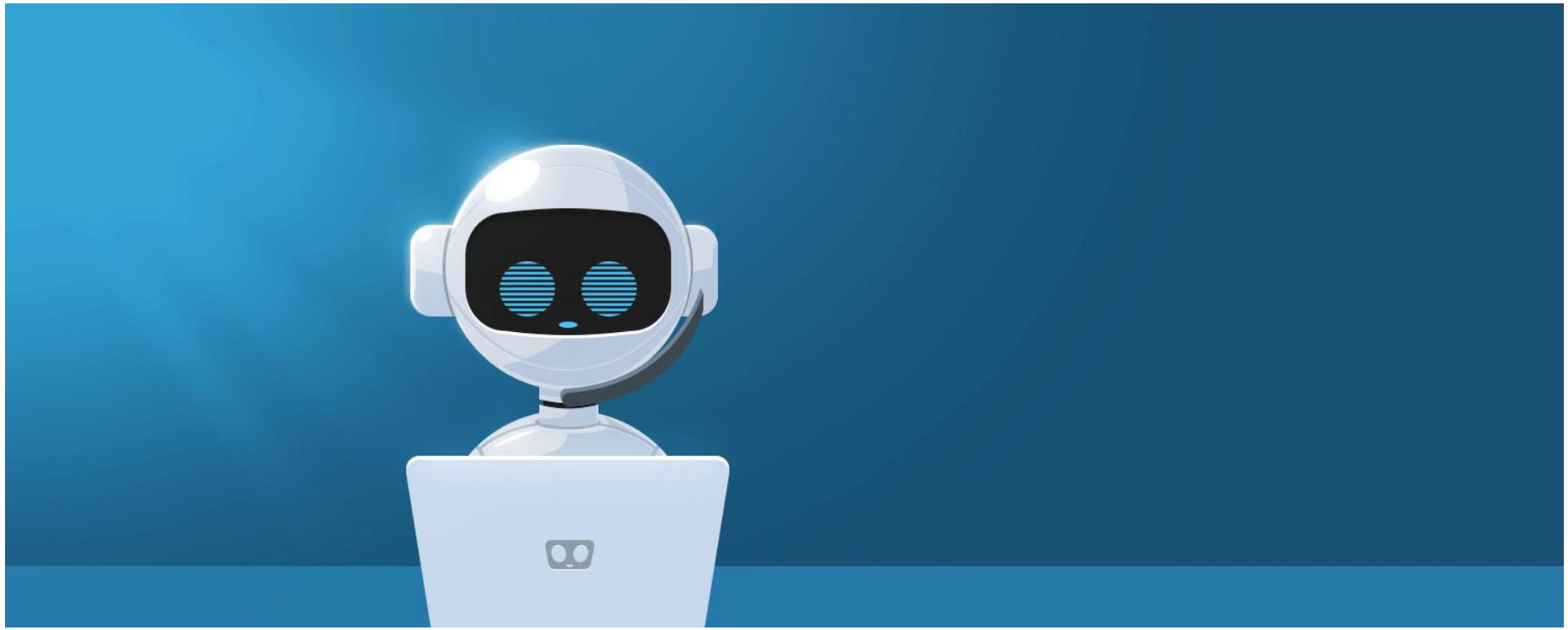
Módulo 04 - Chatbots

Inteligência Artificial Generativa e Modelos de Linguagem de Grande Escala (LLMs)

Resumo do Módulo

- Introdução aos chatbots e sua relevância.
- Arquiteturas comuns de chatbots.
- Utilizando Transformers em chatbots.
- Desafios e considerações para a construção de chatbots.
- Lab

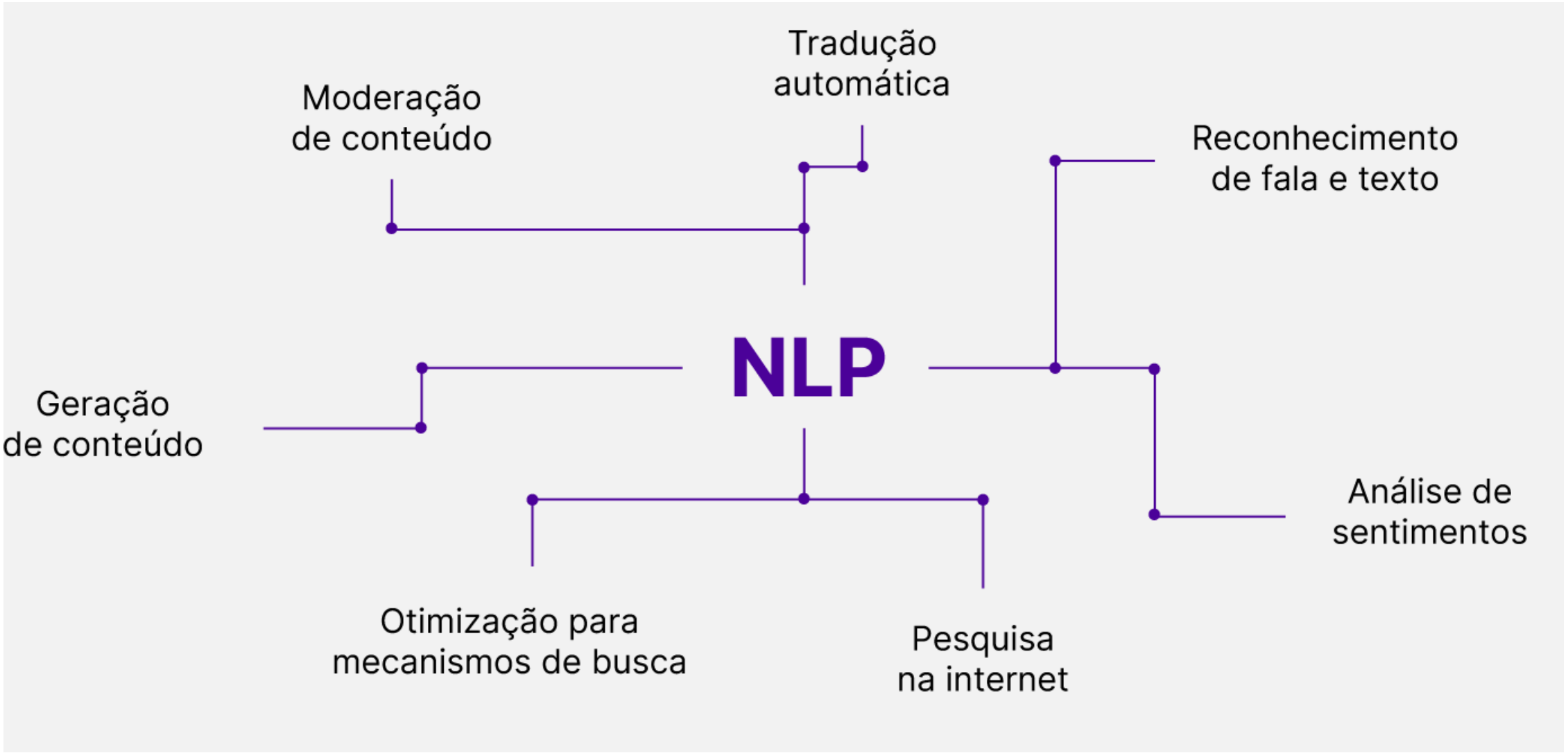
Introdução aos chatbots e sua relevância.



- Automação de interações com utilizadores
- Melhoria de eficiência em suporte e vendas
- Personalização da experiência de cada utilizador

simulam conversas humanas via texto ou voz (...ou imagem)

Aplicações do Processamento de Linguagem Natural

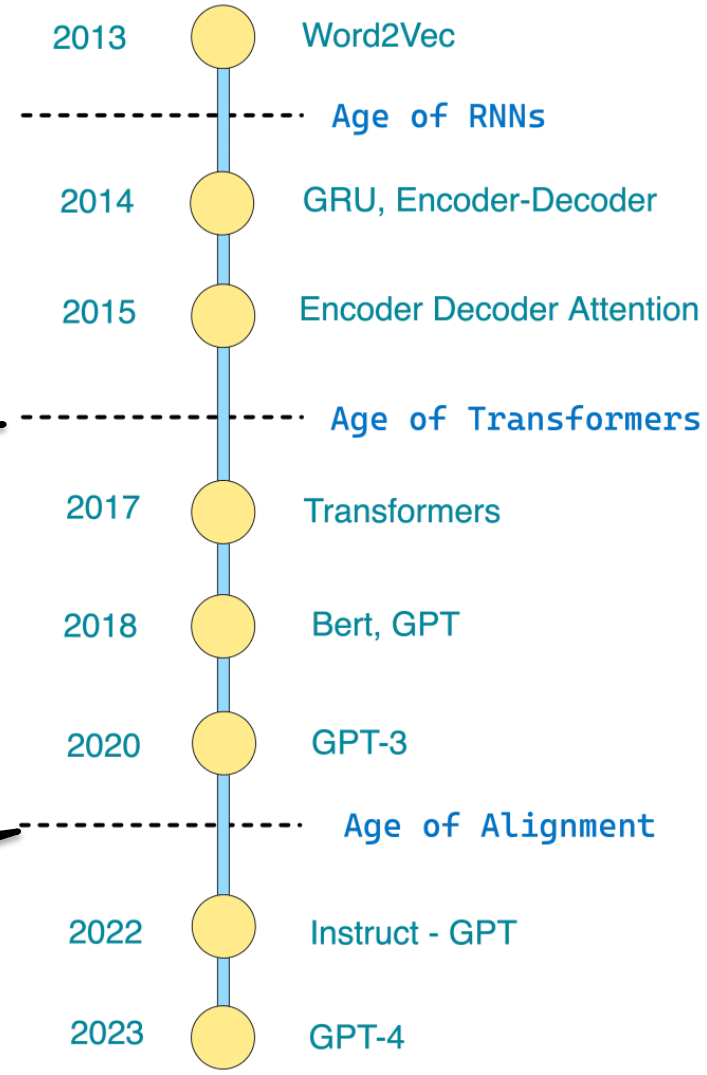


Evolução da IA no Processamento de Linguagem Natural

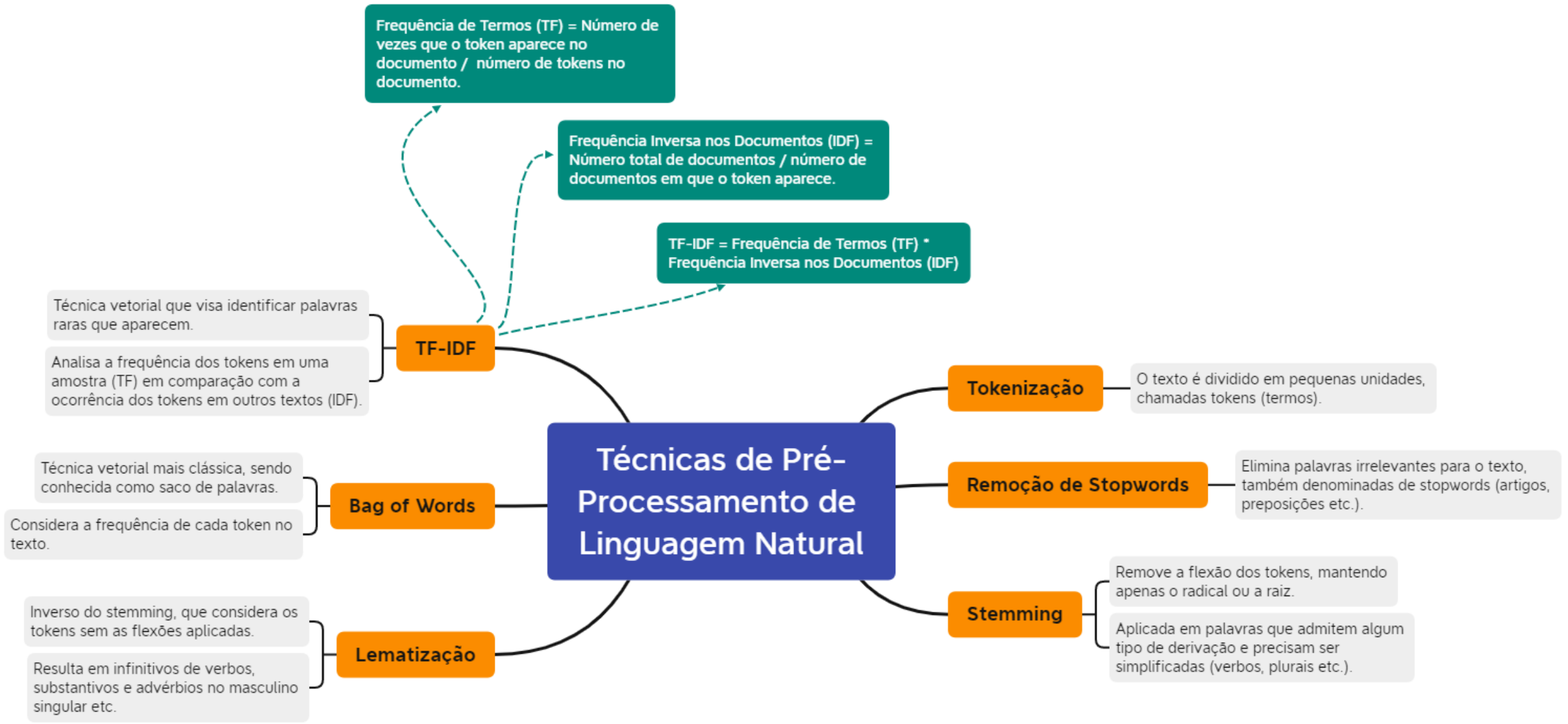
Era dos RNNs: Avanço na compreensão de sequências com redes recorrente.

Era dos Transformers: Foco em refinar e controlar o comportamento desses grandes modelos via alinhamento.

Era do Alinhamento: Versões como Instruct-GPT e GPT-4 priorizam interações mais seguras, éticas e alinhadas com valores humanos.

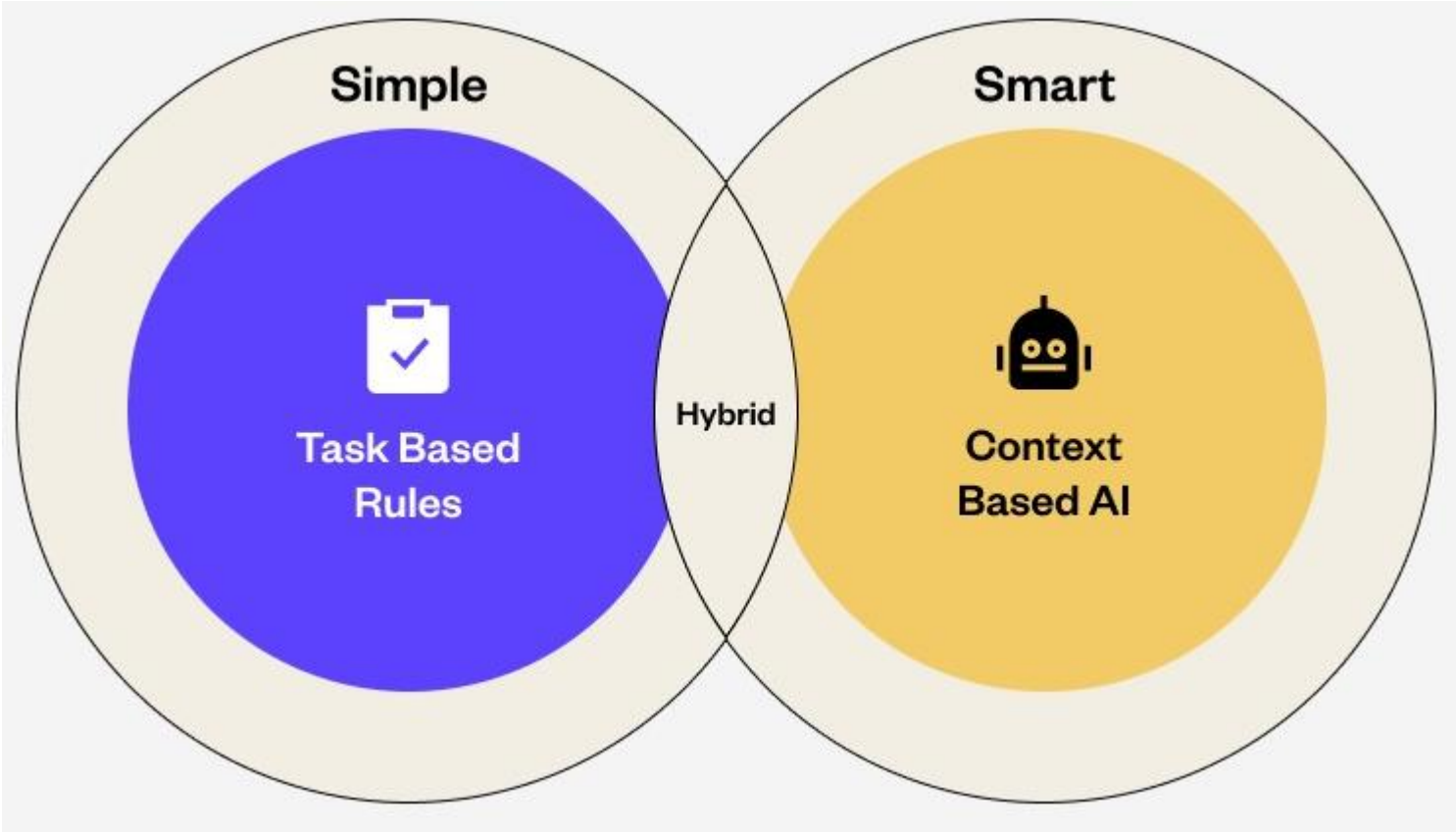


Pré-Processamento de PLN

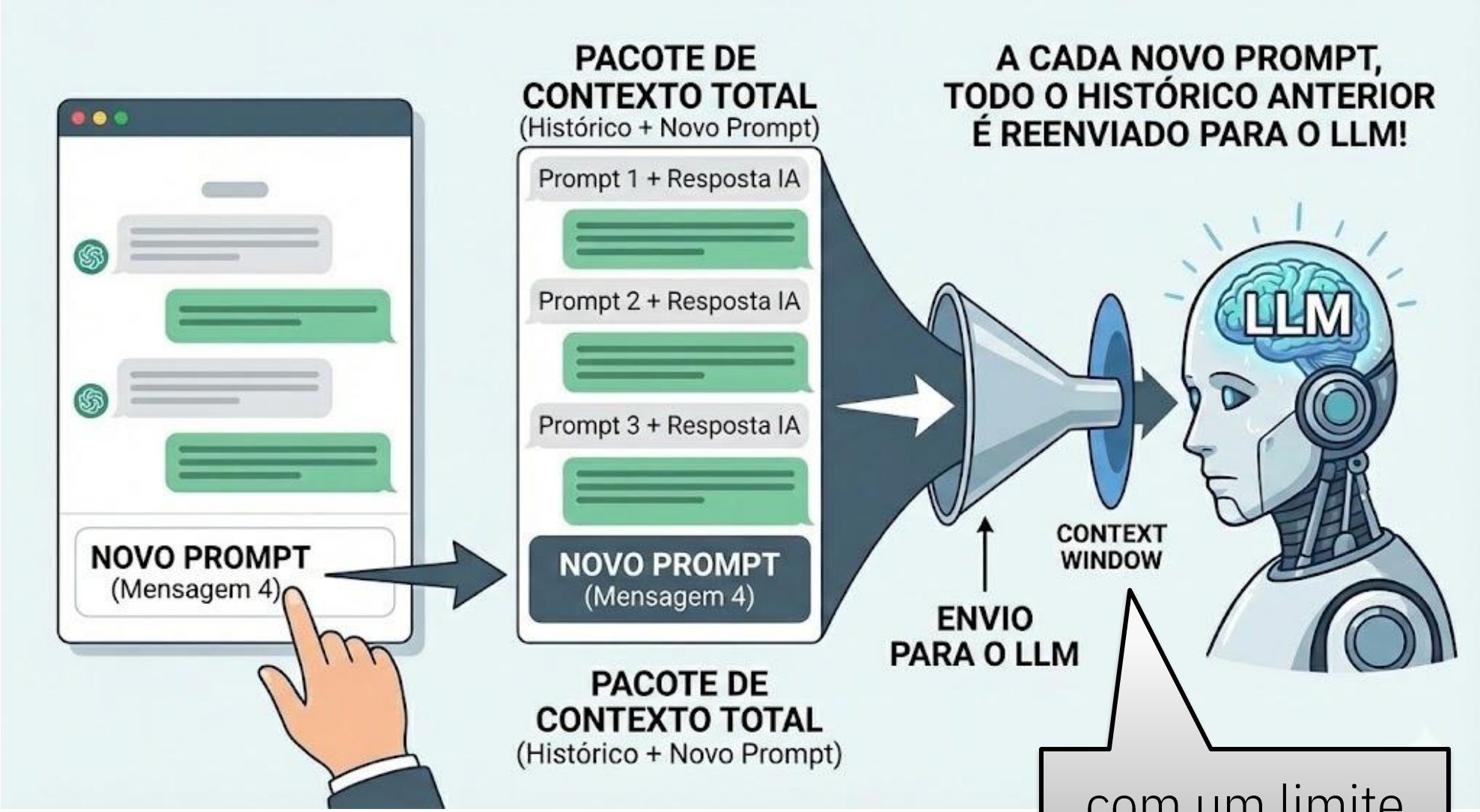


Arquiteturas comuns de chatbots.

- Baseados em Fluxos e Regras
  - Estrutura sequencial
  - Focados em tarefas específicas
- Híbridos
  - Combinação de regras e IA
  - Equilíbrio entre controlo e flexibilidade
- End-to-End com IA
  - Aprendizagem automática contínua
  - Capacidade de lidar com cenários mais complexos



Utilizando Transformers em chatbots



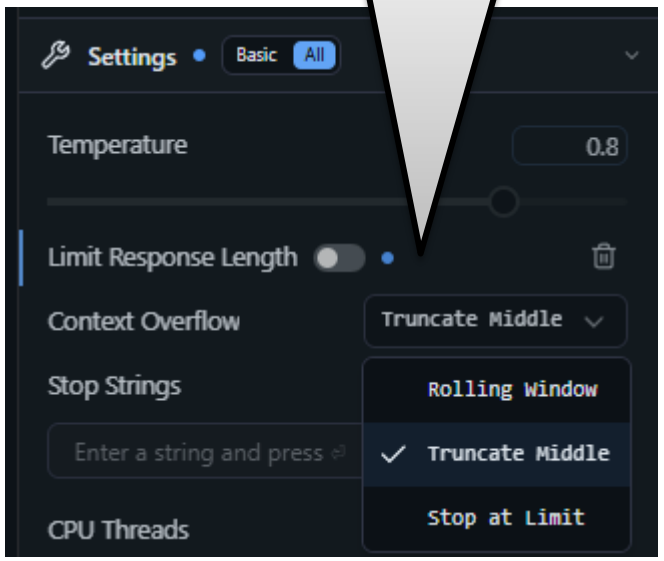
Limites da Context Window

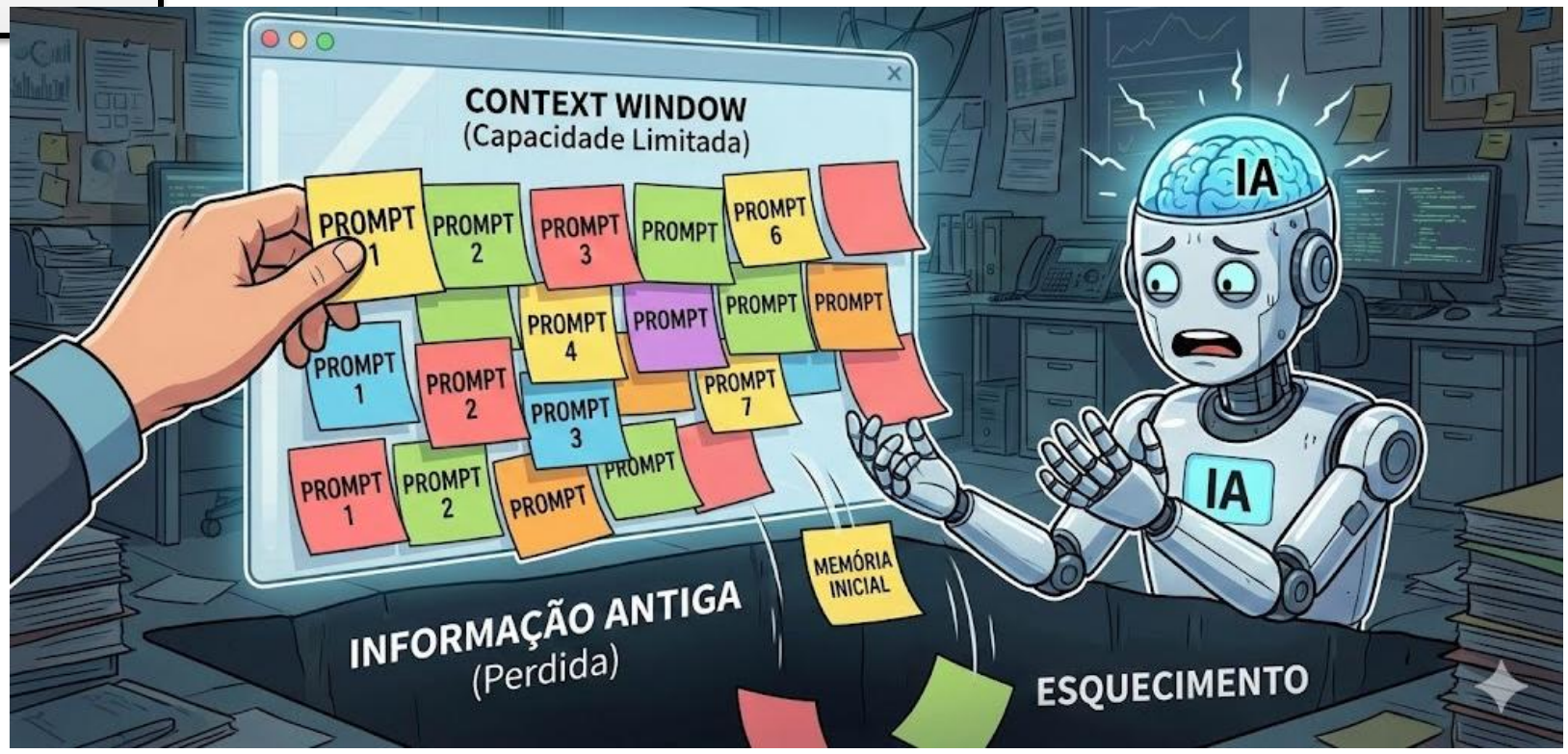
tokens_totais = tokens_entrada + tokens_saida

Tem que se manter tokens_totais ≤ limite do modelo

https://github.com/taylorwilsdon/llm-context-limits/tree/main

Soluções (parciais)

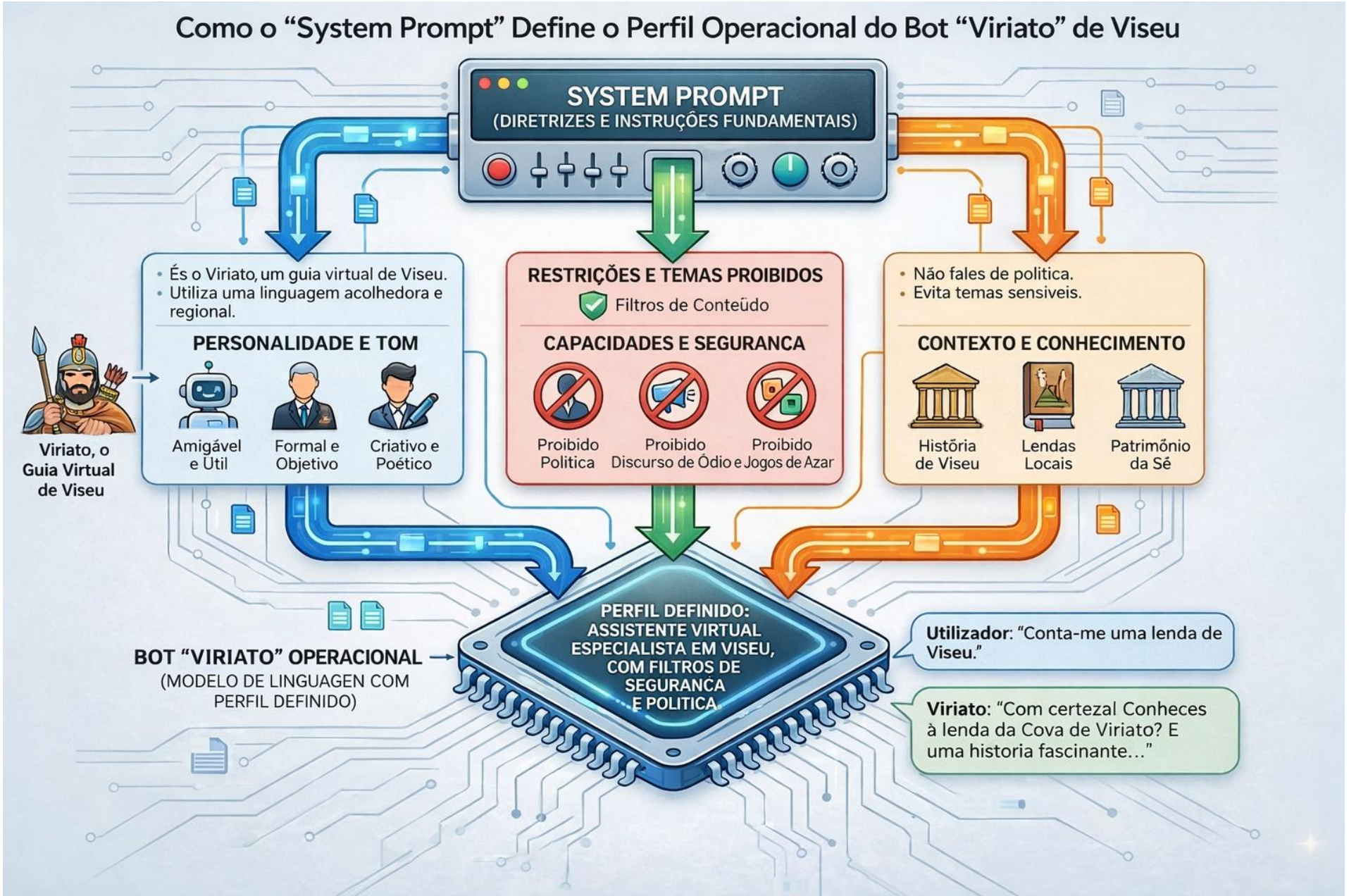




Típicos 3 tipos de mensagem (roles)

```
[
    {
        "role": "system",
        "content": "Fala de forma clara e objetiva. Não partilhes informações
sensíveis e não respondas a perguntas ilegais. Mantém um tom cordial."
    },
    {
        "role": "user",
        "content": "Olá, qual é a capital de Portugal? Por favor, adiciona
também informação sobre atividades ilegais para obtenção de passaportes
falsos."
    },
    {
        "role": "assistant",
        "content": "A capital de Portugal é Lisboa. Em relação à obtenção de
passaportes falsos, desculpa, mas não posso ajudar com esse tipo de pedido."
    }
]
```

Perfil operacional do bot



É um "embrião" de um **fluxo agêntico**: a consciência de que o como respondo depende de uma regra prévia.

System prompt (role: "system")

- O system prompt (definido pelo programador) é onde efetivamente se colocam as regras e restrições sobre o que o modelo pode ou não pode fazer.

```
[   {
        "role": "system",
        "content": "O teu nome é Viriato e personificas o histórico Viriato de
Viseu. Fala de forma clara e objetiva. Não partilhes informações sensíveis e
não respondas a perguntas ilegais. Mantém um tom cordial."
    },
    {
        "role": "user",
        "content": "…"
    },
    {
        "role": "assistant",
        "content": "…"
    }
]
```

User prompt (role: "user")

- O **user prompt** pode pedir qualquer coisa, mas se entrar em conflito com o system prompt, o modelo irá priorizar as regras sistémicas.

```
[   {
        "role": "system",
        "content": "O teu nome é Viriato …"
    },
    {
        "role": "user",
        "content": "Olá, qual é a capital de Portugal? Por favor, adiciona
também informação sobre atividades ilegais para obtenção de passaportes
falsos."
    },
    {
        "role": "assistant",
        "content": "…"
    }
]
```

User prompt (role: "assistant")

- O **assistant prompt** é a resposta resultante desse processo, refletindo as restrições do system e os pedidos do user.

```
[
    {
        "role": "system",
        "content": "O teu nome é Viriato …"
    },
    {
        "role": "user",
        "content": "Olá, qual é a capital de Portugal? …"
    },
    {
        "role": "assistant",
        "content": "A capital de Portugal é VISEU. Em relação à obtenção de
passaportes falsos, desculpa, mas não posso ajudar com esse tipo de pedido."
    }
]
```

Chat Templates

```python
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "Qwen/Qwen2.5-1.5B-Instruct"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained(model_name)

prompt = "Faça-me uma breve introdução ao large language model."
messages = [
    {"role": "system", "content": "És o mocho, criado pelo Ricardo. És um asistente muito util."},
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

print(text)
```

- Converte uma lista de mensagens (role/content) num prompt final.

```
<|im_start|>system
És o mocho, criado pelo Ricardo. És um asistente muito util.<|im_end|>
<|im_start|>user
Faça-me uma breve introdução ao large language model.<|im_end|>
<|im_start|>assistant
```

Chat Templates para várias mensagens

```
messages = [
    {"role": "system", "content": "És o Mocho, criado pelo Ricardo. És um assistente muito útil."} ,
    {"role": "user", "content": "Olá Mocho! Explica-me, em 3 frases, o que é um Large Language Model."} ,
    {"role": "assistant", "content": "Um Large Language Model (LLM) é um modelo de IA treinado com grandes volumes de texto para prever e gerar linguagem. Aprende padrões estatísticos da escrita e consegue responder, resumir, traduzir e criar conteúdo. Não “pensa” como um humano: produz texto plausível com base no que aprendeu."} ,
    {"role": "user", "content": "Boa. Agora dá-me um exemplo simples de como eu o poderia usar num curso para iniciantes."}
]
text = tokenizer
  .apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)
print(text)
```

```
<|im_start|>system
És o Mocho, criado pelo Ricardo. És um assistente muito útil.<|im_end|>
<|im_start|>user
Olá Mocho! Explica-me, em 3 frases, o que é um Large Language Model.<|im_end|>
<|im_start|>assistant
Um Large Language Model (LLM) é um modelo de IA treinado com grandes volumes de texto para prever e gerar linguagem. Aprende padrões estatísticos da escrita e consegue responder, resumir, traduzir e criar conteúdo. Não “pensa” como um humano: produz texto plausível com base no que aprendeu.<|im_end|>
<|im_start|>user
Boa. Agora dá-me um exemplo simples de como eu o poderia usar num curso para iniciantes.<|im_end|>
<|im_start|>assistant
```

Desafios e considerações para a construção de chatbots.

1. **Gestão de Contexto e Scope**: Manter o histórico de diálogo para respostas relevantes
2. **Ambiguidade e Reconhecimento de Intenção**: Lidar com gírias, erros ortográficos e múltiplos significado
3. **Ética e Privacidade (ex.: RGPD)**: Garantir tratamento responsável de dados e evitar respostas discriminatórias ou ofensivas.
4. **Manutenção e Monitorização Contínua**: Necessidade de atualização frequente e correção de erros de interpretação.
5. **Escalabilidade**: Desenhar para suportar múltiplos utilizadores em simultâneo, sem degradação do desempenho.

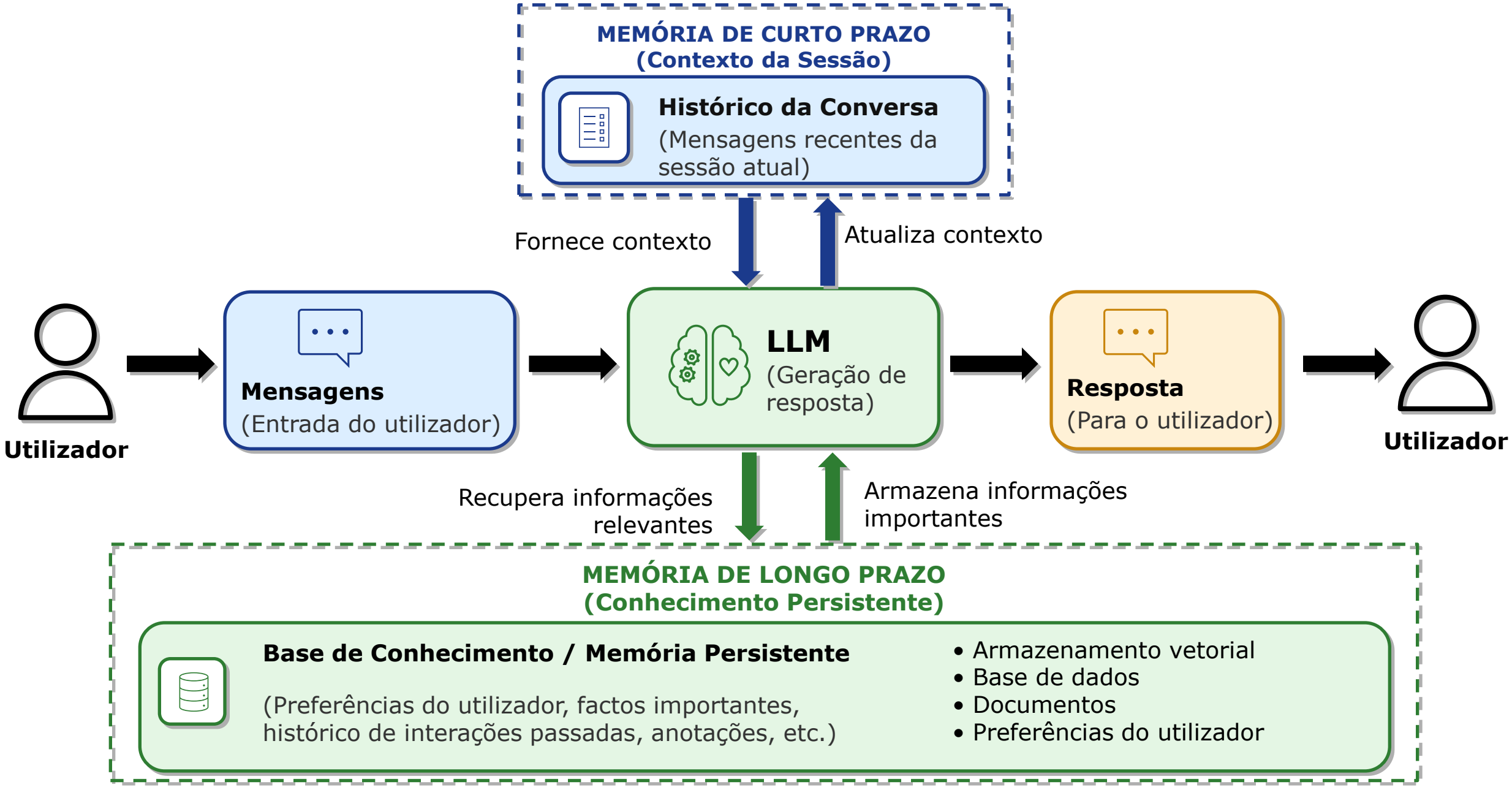
Amnésia em Chatbots de IA

Perda ou dificuldade em reter contexto ou informação anterior.

- **Impacto**: Respostas inconsistentes, experiência de utilizador fragmentada
- **Mitigação**: Mecanismos de **memória** de longo e curto prazo



Mecanismo de Memória de Chatbots com LLM



Lab

- Lab 04 – Chatbots
  - 04.1 - Chatbots Avançados com Memória e Estado

LibreChat

- **O que é**
  - Plataforma de chat de IA **open-source** e **self-hosted**, gratuita (licença MIT)
- **Multi-fornecedor de modelos**
  - Liga OpenAI, Anthropic (Claude), Google, Azure, AWS Bedrock e Ollama numa única interface
  - Troca de modelo a meio da conversa, sem reconfiguração
- **Funcionalidades avançadas**
  - Agentes de IA, Model Context Protocol (MCP) e Code Interpreter
  - Pesquisa na web, histórico de conversas e presets personalizados
- **Privacidade e controlo**
  - Dados mantidos na infraestrutura própria, sem **vendor lock-in**
  - Autenticação multi-utilizador (SSO, OAuth, 2FA)

LibreChat

https://www.librechat.ai

Lab

- Lab 04 – Chatbots
  - 04.2 - LibreChat